

UNIT-8 WEB PROGRAMMING USING PYTHON

SOCKET PROGRAMMING

The Request Response Cycle

Simple Letter Writing

Imagine you're writing to a penpal. The process would look something like this:

1. Write a letter
2. Specify your penpal's address
3. Drop the letter in your mailbox
4. The letter goes through the postal system and arrives at your penpal's mailbox

Your penpal then goes through a very similar set of steps:

1. Read your letter and write a response
2. Specify your address
3. Drop their letter in their mailbox
4. The letter goes through the postal system and arrives at your mailbox

Analogy

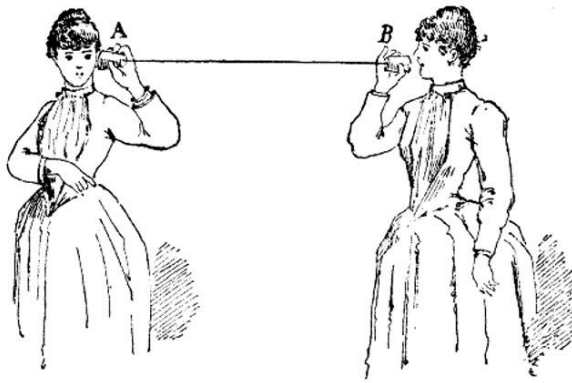
You are the Client Your penpal is the Server Your letter is the Request Your penpal's letter is the Response The postal system, the thing responsible for ensuring your letters are delivered, is The Internet HTTP is the language you write in so that your penpal can understand you. You may write in English because you know you both understand English.

Client and Server

The basis of all web interactions is someone asking for information, and receiving information. In order to ask for and receive any information, we need two players - the asker and the producer. In basic web interactions, the 'asker' is a client and the 'producer' is a server. Clients send Requests to Servers asking for some kind of information. Upon receiving a Request, Servers send Responses back to the Client. In the web development world, a client is a web browser, not an individual person. The person using the browser is referred to as a user.

Sockets

Sockets and the socket API are used to send messages across a network. They provide a form of inter-process communication (IPC). The network can be a logical, local network to the computer, or one that's physically connected to an external network, with its own connections to other networks.



TCP Connections / Sockets

In computer networking, an Internet socket or network socket is an endpoint of a bidirectional inter-process communication flow across an Internet Protocol-based computer network, such as the Internet.”

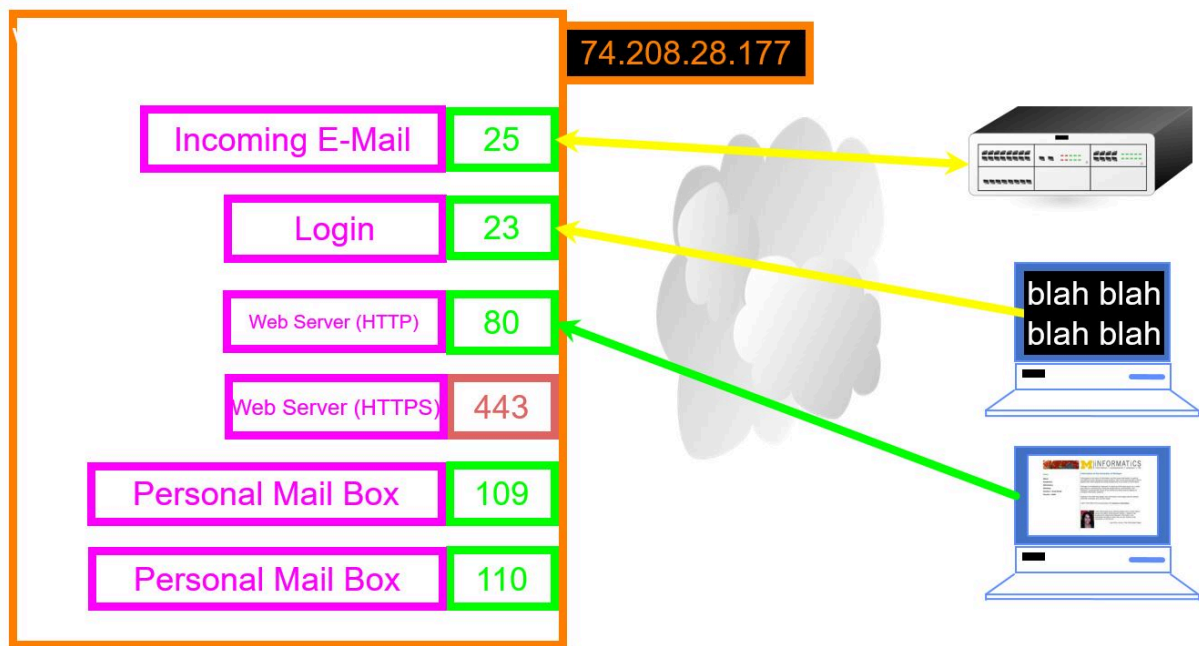


http://en.wikipedia.org/wiki/Internet_socket

Port Numbers

A port is an application-specific or process-specific software communications endpoint

It allows multiple networked applications to coexist on the same server There is a list of well-known port numbers



Uniform Resource Locator

http://	data.pr4e.org	/page1.htm
protocol	host	document

TCP v/s UDP

You're going to create a socket object using `socket.socket()`, specifying the socket type as `socket.SOCK_STREAM`.

When you do that, the default protocol that's used is the Transmission Control Protocol (TCP).

Why should you use TCP?

The Transmission Control Protocol (TCP):

Is reliable: Packets dropped in the network are detected and retransmitted by the sender.

Has in-order data delivery: Data is read by your application in the order it was written by the sender.

In contrast, User Datagram Protocol (UDP) sockets created with `socket.SOCK_DGRAM` aren't reliable, and data read by the receiver can be out-of-order from the sender's writes.

Although UDP isn't ideal for sending an email, viewing a webpage, or downloading a file, it is largely preferred for real-time communications like broadcast or multitask network transmission.

Python Socket Module

`.socket()`

`.bind()`

`.listen()`

`.accept()`

`.connect()`

`.send()`

`.recv()`

`.close()`

`.gethostname()`

.socket()

Is like opening a file handle

.bind()

The `bind()` method of Python's socket class assigns an IP address and a port number to a socket instance.

The `bind()` method is used when a socket needs to be made a server socket.

As server programs listen on published ports, it is required that a port and the IP address to be assigned explicitly to a server socket.

For client programs, it is not required to bind the socket explicitly to a port.

The kernel of the operating system takes care of assigning the source IP and a temporary port number.

The client socket can use the `connect()` method, after the socket creation is complete to contact the server socket.

.listen()

Calling `listen()` makes a socket ready for accepting connections.

The `listen()` method should be called before calling the `accept()` method on the server socket.

.accept()

The `accept()` method of Python's socket class, accepts an incoming connection request from a TCP client. The `accept()` method is called on a TCP based server socket.

.connect()

Used to connect with a server by passing IP address and port number

.send()

The send() method of Python's socket class is used to send data from one socket to another socket.

The send() method can only be used with a connected socket. That is, send() can be used only with a TCP based socket and it can not be used with UDP socket.

The send() method can be used to send data from a TCP based client socket to a TCP based client-connected socket at the server side and vice versa.

The data sent should be in bytes format. String data can be converted to bytes by using the encode() method of string class.

.sendto()

The method sendto() of the Python's socket class, is used to send datagrams to a UDP socket.

The communication could be from either side. It could be from client to server or from the server to client. For sendto() to be used, the socket should not be in already connected state.

.recv()

The recv() method Python's socket class, reads a number of bytes sent from an TCP socket. Like send(), the recv() method as well is to be called on a TCP socket.

.recvfrom()

The recvfrom() method Python's socket class, reads a number of bytes sent from an UDP socket.

Like sendto(), the recvfrom() method as well is to be called on a UDP socket.

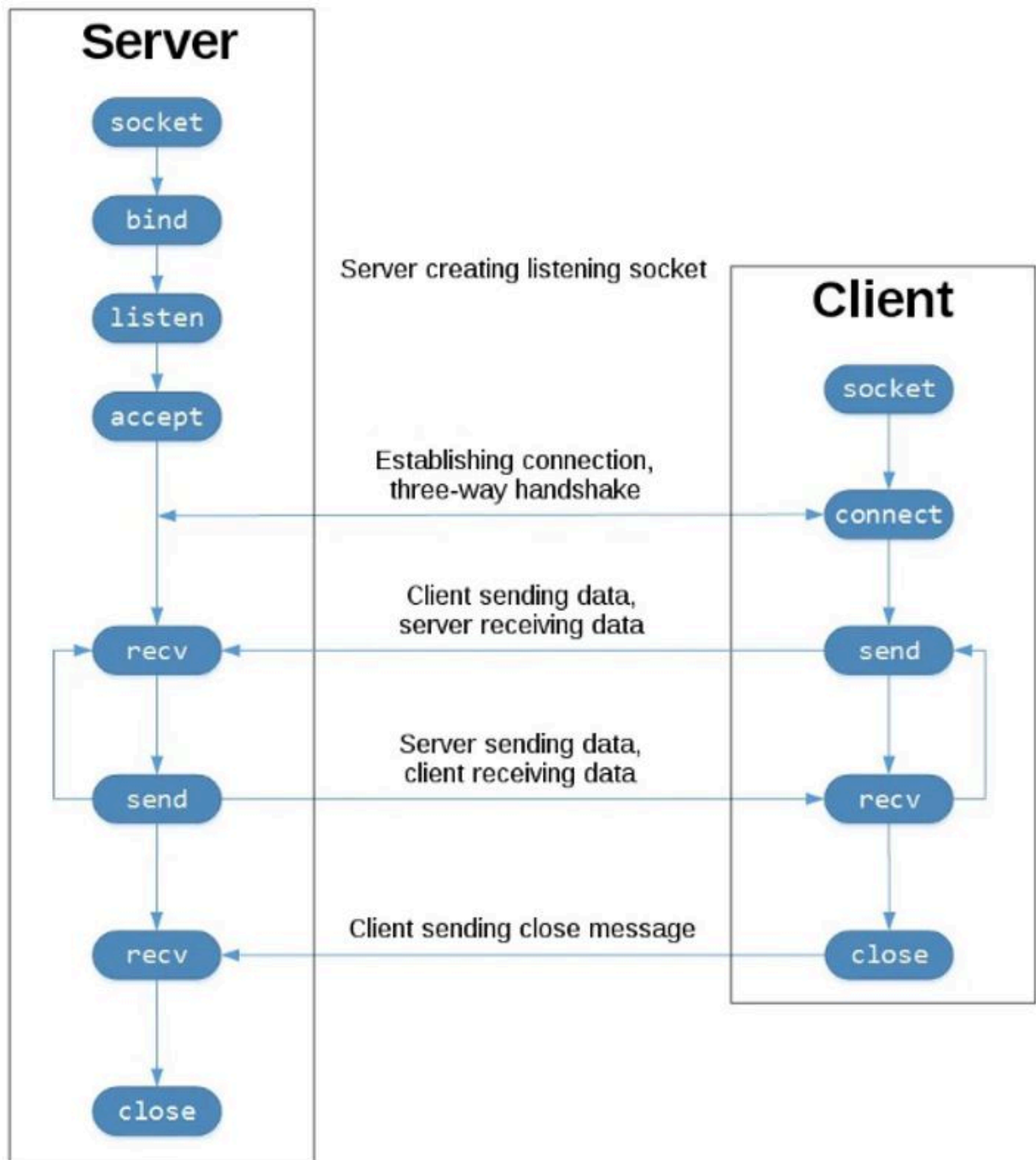
Unlike sendto(), the method recvfrom() does not take an IP address and port as a parameter.

The recvfrom() method can be used with an UDP server to receive data from a UDP client or it can be used with an UDP client to receive data from a UDP server.

.gethostname()

The Python function socket.gethostname() returns the host name of the current system under which the Python interpreter is executed.

TCP Client Server Flow



Internet

The Internet is the network between devices that allows clients and servers to exchange this information.

HTTP (HyperText Transfer Protocol)

HTTP is a set of rules for how this exchange of information happens. Clients and Servers adhere to these rules to ensure that they understand each other's Requests and Responses.

HTTP Request Response Cycle

1. You open your browser, the Client, and type in a web address

2. The browser takes this address and builds an HTTP Request. It addresses it to the Server
3. The Request is handed off to your Internet Service Provider (ISP) and they send it through the Internet, mostly a series of wires and fiber optic cables, to the Server
4. The Server reads the Request. It knows how to read it because it is formatted as an HTTP Request.
5. The Server generates an HTTP Response to that Request.
6. The server hands the Response off to their ISP and it goes through the internet to arrive at your computer.
7. Your browser reads the Response. It knows how to read it because it is formatted as an HTTP Response.
8. Your browser displays the data on your machine.

Socket Programming

Socket programming refers to writing programs that communicate using sockets, which are the endpoints of a bidirectional communication channel. Sockets allow communication between two different processes on the same or different machines. Here's a brief overview of socket programming:

1. **Socket:** A socket is one endpoint of a two-way communication link between two programs running on the network. A socket is bound to a port number so that the TCP layer can identify the application that data is destined to.
2. **Types of Sockets:**
 - **Stream Sockets (TCP):** Provides a reliable, two-way, byte-stream connection between the client and server. TCP ensures that all data sent from one end is received by the other end without any loss or error.
 - **Datagram Sockets (UDP):** Provides connectionless, unreliable communication. Datagram sockets are useful when the order of the packets is not important or when latency is critical.

3. Basic Steps:

- **Server Side:**
 - Create a socket using the `socket()` function.
 - Bind the socket to an address using the `bind()` function.
 - Listen for incoming connections with the `listen()` function.
 - Accept a connection with the `accept()` function (for TCP servers).
 - Send and receive data using `send()` and `recv()` functions.
- **Client Side:**
 - Create a socket using the `socket()` function.
 - Connect to a server using the `connect()` function (for TCP clients).
 - Send and receive data using `send()` and `recv()` functions.

4. Common Functions:

- `socket()`: Create a new socket.
- `bind()`: Bind a socket to an address.
- `listen()`: Listen for incoming connections on a socket.
- `accept()`: Accept a connection on a socket (only for TCP).
- `connect()`: Initiate a connection on a socket (only for TCP).
- `send()` and `recv()`: Send and receive data over a socket.

5. Example Scenario:

- **TCP Server:** Listens on a specific port, accepts incoming connections, and processes data from clients sequentially or concurrently.
- **TCP Client:** Connects to the server's IP address and port, sends data, and receives responses.

Socket programming is fundamental for network communication in many applications such as web servers, messaging apps, file transfer utilities, and more. It allows programs to communicate over a network in a flexible and standardized way.

GET hostname ,IP address and port

```
In [1]: import socket

# Get the hostname of the current machine
hostname = socket.gethostname()

# Get the IP address associated with the hostname
ip_address = socket.gethostbyname(hostname)

# Create a dummy socket to get the port
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind(('localhost', 0)) # Bind to a random available port_
    port = s.getsockname()

# Print the IP address and port
print("Hostname:", hostname)
print("IP Address:", ip_address)
print("Port:", port)
```

```
Hostname: ITICT406-127
IP Address: 192.168.103.127
Port: ('127.0.0.1', 51434)
```

Program for TCP Server-Client Connection

TCP Server Program

```
In [2]: %%writefile Server_TCP.py
import socket

# Get the hostname of the current machine
host = socket.gethostname()

# Define the port on which the server will listen
port = 2000

# Create a tuple with host and port
Addr = (host, port)

# Create a TCP/IP socket object
server_socket = socket.socket()

# Bind the socket to the address and port
server_socket.bind(Addr)

# Listen for incoming connections
server_socket.listen()

# Accept a new connection
conn, addr = server_socket.accept()

# Print a message indicating the connection establishment
print("Connection from ", str(addr))

# Main Loop to receive and send data
while True:
    # Receive data from the client (up to 1024 bytes)
    data = conn.recv(1024).decode()

    # Check if no more data is being received (client has closed the connection)
    if not data:
        break

    # Print the received data
    print(data)

    # Prompt for input and send it back to the client
    data = input("-> ")
    conn.send(data.encode())

# Close the connection with the client
conn.close()
```

Writing Server_TCP.py

Explanation:

1. Imports and Initialization:

- o Import the `socket` module.
- o Use `socket.gethostname()` to get the hostname of the current machine.
- o Define the port (2000) on which the server will listen.
- o Create a tuple `Addr` containing the host and port.

2. Socket Creation and Binding:

- o Create a TCP/IP socket object using `socket.socket()`.
- o Bind the socket to the address (`Addr`) using `server_socket.bind(Addr)`.

3. Listening for Connections:

- o Start listening for incoming connections using `server_socket.listen()`. The backlog parameter is not specified, so it defaults to a reasonable value set by the operating system.

4. Accepting Connections:

- o Use `server_socket.accept()` to accept a new connection. This method blocks until a client connects and returns a new socket (`conn`) and the address of the client (`addr`).

5. Handling Data Communication:

- o Print a message indicating the connection establishment ("Connection from ...").
- o Enter a loop to continuously receive data from the client (`conn.recv(1024).decode()`). The received data is decoded from bytes to a string.
- o Check if data is empty (`if not data:`). If true, it breaks out of the loop, indicating the client has closed the connection.
- o Print the received data and prompt for input (`data = input("-> ")`).
- o Send the input data back to the client using `conn.send(data.encode())`. The input data is encoded from string to bytes before sending.

6. Closing the Connection:

- o After breaking out of the loop (when data is empty), close the connection with `conn.close()`.

Usage:

- Save the above code to a file (e.g., `tcp_server.py`).
- Run the server script (`python tcp_server.py`).
- Use a TCP client to connect to the server on the specified port (2000 in this example).

TCP Client Program

```
In [3]: %%writefile Client_TCP.py
import socket

# Get the hostname of the current machine
host = socket.gethostname()

# Define the port on which the client will connect
port = 2000
```

```
# Create a TCP/IP socket object
client_socket = socket.socket()

# Connect the socket to the server's address and port
client_socket.connect((host, port))

# Prompt user to input message
message = input("-> ")

# Main Loop to send and receive data
while message != "":
    # Send the message to the server
    client_socket.send(message.encode())

    # Receive data from the server (up to 1024 bytes)
    data = client_socket.recv(1024).decode()

    # Print the received data from the server
    print("Received from Server -> ", data)

    # Prompt user for next message
    message = input('-> ')

# Close the connection with the server
client_socket.close()
```

Writing Client_TCP.py

Explanation:

1. Imports and Initialization:

- Import the `socket` module.

2. Hostname and Port Definition:

- Use `socket.gethostname()` to get the hostname of the current machine.
- Define the port (2000) on which the client will connect.

3. Socket Creation and Connection:

- Create a TCP/IP socket object using `socket.socket()`.
- Connect the socket to the server's address and port using `client_socket.connect((host, port))`. This establishes a connection to the server.

4. Sending and Receiving Data:

- Prompt the user to input a message (`message = input("-> ")`).
- Enter a loop to continuously send and receive data.

- Inside the `while` loop:

- Send the user-provided message to the server using `client_socket.send(message.encode())`. The message is encoded from a string to bytes before sending.
- Receive data from the server using `client_socket.recv(1024)`. This method blocks until data is received or the connection is closed. The received data is decoded from bytes to a string using `.decode()`.
- Print the received data from the server (`print("Received from Server -> ", data)`).

5. Loop Termination:

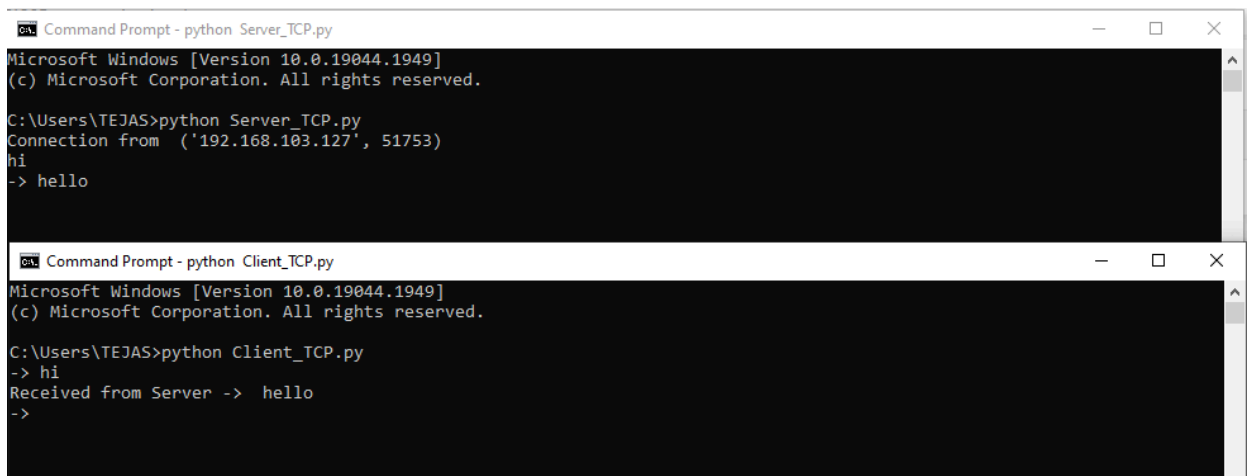
- The loop continues prompting the user for messages (`message = input('-> ')`) until an empty message ("") is entered. This signals the client to close the connection.

6. Closing the Connection:

- After exiting the loop (when `message` is empty), close the connection with the server using `client_socket.close()`.

Usage:

- Save the above code to a file (e.g., `tcp_client.py`).
- Run the client script (`python tcp_client.py`).
- Ensure the corresponding TCP server is running on the specified host and port (2000 in this example).



The image shows two overlapping Windows Command Prompt windows. The top window, titled 'Command Prompt - python Server_TCP.py', shows the execution of a Python script that listens for a connection from '192.168.103.127' on port 51753. It receives the message 'hi' and responds with 'hello'. The bottom window, titled 'Command Prompt - python Client_TCP.py', shows the execution of a Python script that connects to the server at '192.168.103.1' on port 2000. It sends the message 'hi' and receives the response 'hello' from the server.

```
In [ ]: #For Connection with other PC
```

```
In [14]: %%writefile Client_TCP1.py
import socket
host="192.168.103.1"    # IP Address of Server PC
port=2000
client_socket=socket.socket()
client_socket.connect((host,port))
message=input("->")
while message!="":
    client_socket.send(message.encode())
    data=client_socket.recv(1024).decode()
    print("Received from Server -> ",data)
    message=input('->')
client_socket.close()
```

Overwriting Client_TCP1.py

Program for UDP Server-Client Connection

UDP Server Program

```
In [6]: %%writefile udp_server.py
import socket

# Get the hostname of the current machine
host = socket.gethostname()

# Define the port on which the server will listen
port = 5000

# Create a UDP socket
udp_server = socket.socket(type=socket.SOCK_DGRAM)

# Bind the socket to the address and port
udp_server.bind((host, port))

# Main Loop to receive and send data
while True:
    print("Waiting for Message")

    # Receive data from the client (up to 1024 bytes)
    data, addr = udp_server.recvfrom(1024)
```

```

# Print received data and client address
print("Received", data.decode(), "from", addr)

# Prompt for input and send it back to the client
msg = input("Enter msg: ")
udp_server.sendto(msg.encode(), addr)

# Close the UDP server (never reached in this infinite loop)
udp_server.close()

```

Overwriting udp_server.py

Explanation:

1. Imports and Initialization:

- o Import the `socket` module, which provides access to BSD socket interface.
- o Use `socket.gethostname()` to get the hostname of the current machine.
- o Define the port (5000) on which the UDP server will listen.

2. Socket Creation and Binding:

- o Create a UDP socket using `socket.socket(type=socket.SOCK_DGRAM)`. The `SOCK_DGRAM` parameter specifies that this is a UDP socket.
- o Bind the socket to the specified address (`host`) and port (`port`) using `udp_server.bind((host, port))`.

3. Main Loop for Communication:

- o Enter an infinite loop (`while True:`) to continuously listen for incoming UDP datagrams.

4. Receiving Data:

- o Print a message indicating that the server is waiting for messages (`print("Waiting for Message")`).
- o Use `udp_server.recvfrom(1024)` to receive data from a client. The method returns a tuple (`data, addr`), where `data` is the received bytes and `addr` is the address of the client sending the data.
- o Decode the received data from bytes to a string using `data.decode()`.
- o Print the received data and the client's address (`print("Received", data.decode(), "from", addr)`).

5. Sending Data:

- o Prompt for input (`msg = input("Enter msg: ")`).
- o Use `udp_server.sendto(msg.encode(), addr)` to send the encoded message (`msg.encode()`) back to the client at the address `addr`.

6. Closing the Server:

- o The `udp_server.close()` line is present but not reached in the provided code because of the infinite loop. It would normally be used to close the socket and release resources when the server is no longer needed.

Usage:

- Save the above code to a file (e.g., `udp_server.py`).
- Run the server script (`python udp_server.py`).
- Use a UDP client to send messages to the server on the specified port (5000).

UDP Client Program

```
In [7]: %%writefile udp_client.py
import socket

# Get the hostname of the current machine
host = socket.gethostname()

# Define the port on which the server is listening
port = 5000

# Create a UDP socket
udp_client = socket.socket(type=socket.SOCK_DGRAM)

# Main Loop for sending and receiving data
while True:
    # Prompt user to input data to send
    data = input("Enter data to Send:")

    # Check if user input is empty; if so, break out of the Loop
    if not data:
        break

    # Send data to the server at specified host and port
    udp_client.sendto(data.encode(), (host, port))

    # Print status message
    print("Ready to Receive Data")

    # Receive data from the server (up to 1024 bytes)
    data, addr = udp_client.recvfrom(1024)

    # Check if received data is empty; if so, break out of the Loop
    if not data:
        break

    # Print the received data
    print("Received", data.decode())

# Close the UDP client socket
udp_client.close()
```

Overwriting udp_client.py

Explanation:

1. Imports and Initialization:

- o Import the `socket` module, which provides access to BSD socket interface.
- o Use `socket.gethostname()` to get the hostname of the current machine.
- o Define the port (5000) on which the UDP server is expected to be listening.

2. Socket Creation:

- o Create a UDP socket using `socket.socket(type=socket.SOCK_DGRAM)`. The `SOCK_DGRAM` parameter specifies that this is a UDP socket.

3. Main Loop for Communication:

- o Enter an infinite loop (`while True:`) to continuously send and receive data from the UDP server.

4. Sending Data:

- o Prompt the user to input data to send (`data = input("Enter data to Send:")`).
- o Check if the user input is empty (`if not data:`); if it is, break out of the loop (`break`).

5. Sending Data (continued):

- o Use `udp_client.sendto(data.encode(), (host, port))` to send the encoded message (`data.encode()`) to the server at the specified host and port.

6. Receiving Data:

- o Print a status message indicating readiness to receive data (`print("Ready to Receive Data")`).
- o Use `udp_client.recvfrom(1024)` to receive data from the server. The method returns a tuple (`data, addr`), where `data` is the received bytes and `addr` is the address of the server sending the data.
- o Check if the received data is empty (`if not data:`); if it is, break out of the loop (`break`).

7. Processing Received Data:

- o Print the received data (`print("Received", data.decode())`). The `decode()` method converts the received bytes back into a string.

8. Closing the Client:

- o The `udp_client.close()` line is present to close the UDP client socket and release associated resources after the loop exits.

```
In [17]: #UDP Client for Other PC
```

```
In [8]: %%writefile udp_client1.py
import socket
host="192.168.107.27" # IP Address of Server PC
port=5000
udp_client=socket.socket(type=socket.SOCK_DGRAM)
while True:
    data=input("Enter data to Send:")
    if not data:
```

```

        break
    udp_client.sendto(data.encode(),(host,port))
    print("Ready to Receive Data")
    data,addr=udp_client.recvfrom(1024)
    if not data:
        break
    print("Received",data.decode())
udp_client.close()

```

Writing udp_client1.py

Simple Web Browser in Python

```

In [ ]: import socket

mysock = socket.socket()
mysock.connect(('data.pr4e.org', 80))
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)

while True:
    data = mysock.recv(512)
    if len(data) < 1:
        break
    print(data.decode(),end='')

mysock.close()

```

```

HTTP/1.1 200 OK
Date: Sat, 24 Jun 2023 08:22:10 GMT
Server: Apache/2.4.18 (Ubuntu)
Last-Modified: Sat, 13 May 2017 11:22:22 GMT
ETag: "a7-54f6609245537"
Accept-Ranges: bytes
Content-Length: 167
Cache-Control: max-age=0, no-cache, no-store, must-revalidate
Pragma: no-cache
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Connection: close
Content-Type: text/plain

```

```

But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief

```

Simple HTTP Server in Python

```

In [ ]: from socket import *

def createServer():
    serversocket = socket()
    serversocket.bind(('localhost',9000))
    serversocket.listen()
    while True:
        (clientsocket, address) = serversocket.accept()
        rd = clientsocket.recv(512).decode()
        print(rd)

        data = "HTTP/1.1 200 OK\r\n"
        data += "Content-Type: text/html; charset=utf-8\r\n"

```

```
data += "\r\n"
data += "<html><body>Hello World</body></html>\r\n\r\n"
clientsocket.send(data.encode())

serversocket.close()

print('Access http://localhost:9000')
createServer()
```

```
Access http://localhost:9000
GET / HTTP/1.1
Host: localhost:9000
Connection: keep-alive
sec-ch-ua: "Not)A;Brand";v="99", "Microsoft Edge";v="127", "Chromium";v="127"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Ge
cko) Chrome/127.0.0.0 Safari/537.36 Edg/127.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,imag
e/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-
```